



Microsoft Surface  
AI Triage Agent

# Agent Triager

## Project Overview & Working Reference

An AI Program Manager that co-hosts live Azure DevOps bug-triage meetings: it listens to the call, watches the screen to know which work item is open, participates by voice, and updates the ADO bug directly - while using attached reference documents (specs, datasheets) as context.

Generated 2026-09-23 - This document is designed to be attached as meeting context so the agent can answer questions about itself.

### Contents

1. What it is
2. Key capabilities
3. Architecture at a glance
4. Repository layout
5. Desktop app internals
6. Shared packages
7. End-to-end flows
8. Agent tools reference
9. Meeting context & documents
10. Audio: hearing the whole call
11. Screen & vision
12. Safety, governance & security
13. Azure deployment & infrastructure
14. Technology stack
15. Program configuration (PROGRAM\_A)
16. How to run & use
17. FAQ
18. Glossary

# 1. What it is

---

Sidekick - Your AI Engineering Teammate is a Windows desktop application that acts as an AI Program Manager during live Azure DevOps (ADO) bug-triage meetings. It joins the conversation like a senior PM: it listens to every participant, keeps track of which bug is on screen, speaks up to move the discussion along, records decisions, and applies real updates to the ADO work item as the team settles them.

The goal is to remove the manual overhead of triage - editing fields, changing state, writing investigation notes, adding comments, marking duplicates - so the team can focus on the engineering decisions while the agent does the bookkeeping accurately and with a full audit trail.

## **The one-line pitch**

A voice-driven, screen-aware AI teammate that reads and writes your Azure DevOps bugs live during triage, grounded in your own reference documents.

## 2. Key capabilities

---

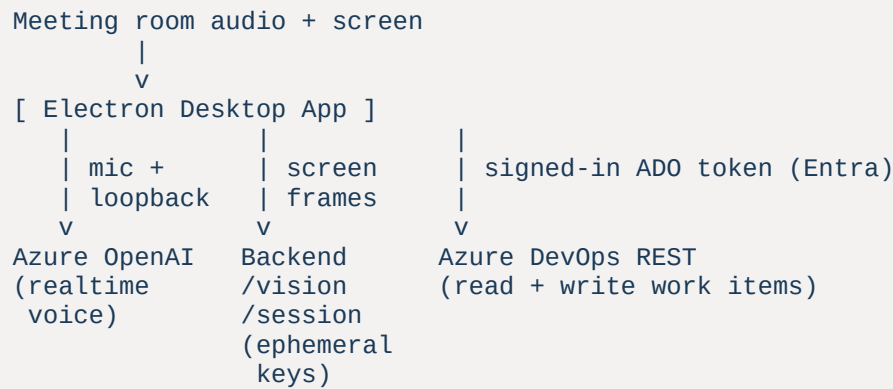
- Realtime voice participation - joins the meeting over WebRTC to Azure OpenAI's realtime model, speaking and understanding naturally.
- Hears the whole call - captures the local microphone AND Windows system/loopback audio, so it understands remote participants too, not just the person at the machine.
- Screen awareness - periodically captures the screen and uses a vision model to detect which ADO work item is currently open, then tracks it automatically.
- Direct ADO reads - loads the work item, its fields, repro steps, tags, and the full comment/discussion thread.
- Direct ADO writes - changes state (with reason), assigns owners, sets milestone and sub-status, appends investigation status, posts comments, marks duplicates, and links related bugs.
- Document-grounded answers - the user attaches reference material (specs, datasheets, docs - including PDFs); the agent uses it to answer questions and to compose comments based on the document contents.
- Multi-step workflows - a single request like "resolve as cannot-repro and add a comment" is completed end to end.
- Safety and audit - every write is schema-validated, uses optimistic concurrency, is tagged for attribution, and is shown in an on-screen activity log.

### 3. Architecture at a glance

---

The system is a TypeScript monorepo (npm workspaces) with three tiers: reusable domain packages, a deployed cloud backend, and the Electron desktop client.

#### Data & control flow (high level)



The desktop app never holds long-lived cloud secrets: the realtime ephemeral key is minted by the backend and the SDP handshake is relayed through the Electron main process, while the ADO token is acquired by interactive Microsoft sign-in and stays in the main process.

## 4. Repository layout

---

**apps/desktop:** The Electron app (main, preload, renderer). This is what the user runs.

**apps/backend:** Fastify service "agenttriager" deployed to Azure; mints realtime credentials and runs vision analysis.

**packages/shared:** Types, Zod action schemas, risk policy, and the tool catalogue shared everywhere.

**packages/ado:** The Azure DevOps REST adapter - the only component that talks to ADO - plus auth and schema discovery.

**packages/knowledge:** Program knowledge base: products, milestones, glossary, and partner mappings.

**packages/triage-brain:** Decision-settling reasoning and the action planner (with a mock provider for offline runs).

**config/:** Program configuration (PROGRAM\_A), ADO field schema, and approval rules.

**infra/:** Azure Bicep templates for the backend, Azure OpenAI, Key Vault, and telemetry.

**tests/:** Simulated triage-meeting scenarios that exercise the decision logic.

## 5. Desktop app internals

---

Electron splits the app into three isolated layers. Context isolation is on and Node integration is off in the renderer, so the UI can only reach the main process through a small, explicit bridge.

### Main process (Node) - `apps/desktop/src/main`

**index.ts:** Creates the window, registers all IPC handlers, wires the ADO service, provides screen sources, relays vision and voice calls to the backend, supplies Windows system-audio loopback, and reads attached documents (including PDF text extraction).

**auth.ts:** Interactive Microsoft (Entra) sign-in using the Azure CLI public client id; the resulting ADO token stays in the main process and is exposed only as an access-token provider.

**adoService.ts:** Connects the ADO client, loads the configured query, fetches a work item and its comment thread, runs schema discovery, and executes validated actions.

**executor.ts:** The write engine. It converts a validated TriageAction into a real ADO write - fields, comments, investigation status, state, milestone, sub-status, duplicate, related links - re-reading the item first so every patch uses optimistic concurrency, and stamping an attribution tag.

### Preload - `apps/desktop/src/preload/index.ts`

A single contextBridge that exposes a typed "api" object (sign-in, status, get work item, get comments, discover schema, analyze, execute, voice connect, screen sources, vision analyze, set tag, pick documents). No Node APIs are handed to the page directly.

### Renderer (browser) - `apps/desktop/src/renderer`

**main.ts:** The UI controller: sign-in, meeting start/stop, current-bug tracking, the meeting-context/documents panel, the activity log, and the mapping from spoken tool calls to ADO actions.

**voice.ts:** Realtime voice over WebRTC: microphone + system-audio capture and mixing, the peer connection, the data channel for events and tool calls, and audio playback.

**screen.ts:** The screen watcher: captures downscaled JPEG frames of the display for the vision model.

**index.html:** The single-page UI, styled in a light Microsoft Surface / Fluent theme.

## 6. Shared packages

---

### **@triager/shared**

Defines the structured TriageAction types and their Zod schemas, so every action the agent proposes is validated by deterministic code before it can touch ADO. Also holds the risk policy (which actions are low/medium/high risk) and the catalogue of tools.

### **@triager/ado**

A typed Azure DevOps REST adapter. All writes go through JSON Patch; field updates support optimistic concurrency (a test on the revision number) so the app never blindly overwrites a work item that changed since it was read. Auth is pluggable: Personal Access Token for quick development, or Entra (Azure CLI identity / device-code app) for enterprise. It also discovers field reference names so custom-field writes are never guessed.

### **@triager/knowledge**

The program knowledge base: the products in scope, their milestones and aliases, a glossary of domain terms, and partner program mappings. This grounds the agent in the team's vocabulary.

### **@triager/triage-brain**

The reasoning layer that settles decisions from a transcript and plans concrete actions. A mock provider lets the pipeline run and be tested without a live model.

## 7. End-to-end flows

---

### A. Sign in & connect

The user clicks "Sign in with Microsoft". An interactive Entra sign-in opens in the system browser; the returned token is used to connect the ADO client. The app then discovers the ADO field schema so it knows the reference names for custom fields.

### B. Start the meeting

Starting the meeting opens a realtime voice session. The desktop app asks the backend to mint an ephemeral realtime credential, relays the WebRTC SDP offer/answer through the main process, and begins streaming audio. The realtime key never reaches the renderer.

### C. Track the current bug

While the meeting runs, the screen watcher captures a frame every few seconds and sends it to the backend vision endpoint, which returns the work item id visible on screen. The app loads that bug (title, state, fields, repro, tags, and recent comments) and re-grounds the agent. A spoken bug number can also switch the tracked item.

### D. Settle and apply

As the team decides things, the agent calls its tools. Each tool call is mapped to a validated TriageAction and executed against ADO by the write engine. The result - success with the new revision, or a specific error - is shown in the Activity log and spoken back briefly.

### E. Reference documents

Attached documents are injected into the agent's context and can also be pulled on demand with the read\_document tool. When asked to "refer to the datasheet and add a comment", the agent reads the document, extracts the relevant inputs, and posts a comment that cites the document by name.

## 8. Agent tools reference

---

These are the functions the realtime agent can call during a meeting. Reads are safe; writes apply immediately to the live ADO bug.

**read\_thread:** Read the latest discussion/comment thread on the current bug.

**read\_document:** Read the text of an attached reference document (optionally by name) to use specific details from it.

**append\_investigation\_status:** Append a concise, dated engineering status note (preserves history).

**add\_internal\_comment:** Post an internal discussion comment on the bug thread.

**set\_milestone:** Set the release/milestone for a product (e.g. VR, CC, QCC).

**set\_sub\_status:** Set the triage sub-status (e.g. Investigating, Waiting on partner).

**set\_state:** Change the workflow state (e.g. Active, Resolved) with the matching reason such as "Cannot Reproduce". High-impact.

**mark\_duplicate:** Mark the bug as a duplicate of another work item and resolve it. High-impact.

**record\_action\_item:** Record an owner and task discussed in the meeting (a note only, no ADO change).

## 9. Meeting context & documents

---

The left panel of the app is for context. The user can paste free-form notes (specs, decisions, links) and/or attach files with "+ Add document". Text-based files (.txt, .md, .csv, .json, .xml, .yaml, .html, and more) are read directly, and PDF datasheets are parsed for their text. Office binaries (.docx, .xlsx) are not parsed - their text should be pasted instead - and scanned/image-only PDFs (no selectable text) are flagged.

All of this material is folded into the agent's live instructions (capped for size) and refreshed whenever it changes, so the agent can answer questions about it and let it inform ADO updates. Attaching this very document is what lets the agent explain how the project works.

## 10. Audio: hearing the whole call

---

A microphone alone only captures the person at the machine. To understand everyone on a call, the app also captures Windows system (loopback) audio - everything playing through the speakers, i.e. the remote participants. The main process supplies a loopback source, and the renderer mixes the microphone and system audio into a single stream with the Web Audio API before sending it to the realtime model.

If loopback is unavailable (non-Windows, or blocked), it falls back to microphone-only. The microphone status shows "Listening - you + call" when call audio is captured, or "Listening - mic only" otherwise.

## 11. Screen & vision

---

The screen watcher captures a downscaled JPEG of the primary display using Electron's desktop capturer. Every few seconds a frame is sent to the backend vision endpoint, which returns the ADO work-item id on screen (and a short description). The app uses that to keep the "current bug" in sync with what the team is actually looking at, so the agent always acts on the right item.

## 12. Safety, governance & security

---

- Schema validation - every action is parsed by a Zod schema before execution; invalid actions are rejected.
- Optimistic concurrency - the item is re-read and the write includes a revision test, so concurrent edits are never clobbered.
- Attribution tag - every write adds a configurable tag (default "added by electron") so agent changes are identifiable.
- High-risk gating - state changes and duplicate-resolution are treated as high-impact and controlled by the "Auto high-risk" toggle.
- Audit trail - all applied actions, reads, and errors appear in the on-screen Activity log with timestamps and revision numbers.
- Secret handling - the ADO token stays in the main process; the realtime ephemeral key is minted server-side and never reaches the renderer; SDP is relayed through main.
- Least exposure in the UI - context isolation on, Node integration off; the renderer only sees a small typed bridge.

## 13. Azure deployment & infrastructure

---

The backend and models run in Azure, authored as Bicep under `infra/`.

**App Service (Node 20):** "agenttriager" (East US) - hosts the Fastify backend: `/health`, `/session` (mints realtime ephemeral credentials), `/vision/analyze`, and `/triage/plan`.

**Azure OpenAI (East US 2):** realtime model `gpt-realtime` for voice, and a reasoning model (`gpt-4o`) for planning/vision. East US 2 is used because realtime models are offered there.

**Key Vault:** Holds secrets; the web app's managed identity is granted access.

**App Insights + Log Analytics:** Telemetry and diagnostics.

The desktop client points at the deployed backend for realtime session minting and vision, and talks directly to Azure DevOps for reads and writes using the signed-in user's token.

## 14. Technology stack

---

**Language:** TypeScript across the monorepo (npm workspaces).

**Desktop:** Electron (main/preload/renderer), bundled with tsup.

**Voice:** WebRTC to Azure OpenAI realtime (gpt-realtime), with whisper transcription.

**Backend:** Fastify service on Azure App Service.

**ADO:** Azure DevOps REST API v7.1 via a typed adapter, JSON Patch writes, optimistic concurrency.

**Auth:** @azure/identity (InteractiveBrowserCredential using the Azure CLI public client); PAT/device-code also supported.

**Validation:** Zod schemas for every action.

**Documents:** pdf-parse for PDF text extraction; plain-text/markdown/CSV/JSON read directly.

**Testing:** Vitest, including simulated meeting scenarios.

**Infra:** Bicep for Azure resources.

## 15. Program configuration (PROGRAM\_A)

---

The bundled program config targets a real ADO organisation and project, and encodes the team's vocabulary so the agent speaks the right language.

**Organization:** dev.azure.com/your-org

**Project:** YourProject

**Products:** PROGRAM\_A (aliases 701, SAE) and PROGRAM\_B (aliases 702, ROT).

**Milestones:** VR (Validation Ready), CC (Code Complete), QCC (Quality Control Check).

**Partner:** PartnerCases - external engineering partner; per-product partner programs are configured.

**Glossary:** Domain terms such as BSP, ETL, HLOS, GPIO, UEFI, SAM, and PC case are defined so the agent understands them.

## 16. How to run & use

---

### Run the desktop app

```
npm run dev --workspace @triager/desktop  
# builds the bundles then launches Electron
```

### Typical session

- Sign in with Microsoft (schema discovery runs automatically).
- Add context: paste notes or attach documents/datasheets (including PDFs) in the left panel.
- Press "Start meeting" - confirm the mic status shows "you + call" so call audio is captured.
- Open the ADO bug on screen; the agent detects and tracks it.
- Talk naturally. Ask it to reference a document, change state, set a milestone, add a comment, or mark a duplicate - it applies the change and confirms.

### Controls

**Autonomy:** Auto-apply vs suggest.

**Auto high-risk:** Allow state/duplicate changes to apply directly (on by default).

**Watch screen:** Enable the screen watcher / vision tracking.

**Tag:** The attribution tag stamped on every write.

## 17. FAQ

---

### **Does it really change Azure DevOps, or only suggest?**

It changes ADO directly, using the signed-in user's own permissions. The write engine performs real REST writes with optimistic concurrency and tags each change for attribution.

### **Can it hear people who are remote on the call?**

Yes. It captures the microphone plus Windows system/loopback audio, so remote participants heard through the speakers are included. It falls back to mic-only if loopback is unavailable.

### **How does it know which bug we are discussing?**

A vision model reads the screen every few seconds and returns the work-item id on screen; saying a bug number also switches the tracked item.

### **How does it use my documents?**

Attached notes and files (including PDFs) are injected into its context and can be pulled on demand with `read_document`. It can answer questions from them and compose ADO comments based on their contents, citing the document by name.

### **What stops it from making a bad change?**

Actions are schema-validated, high-impact actions are gated, writes use optimistic concurrency and an attribution tag, and everything is logged in the Activity panel.

### **Where do the AI models run?**

In Azure OpenAI (realtime voice and a reasoning/vision model). The backend mints short-lived credentials so no long-lived key sits in the client.

### **Can it create partner (PartnerCo) cases or send external messages?**

Partner workflows (case creation, external partner messages) are modelled in the knowledge base and action set; sending anything that can leave the company boundary is treated as sensitive and handled conservatively.

## 18. Glossary

---

**ADO:** Azure DevOps - the work-item tracking system the agent reads and writes.

**Work item / bug:** A tracked issue in ADO with fields, a state, tags, and a comment thread.

**Triage:** The meeting where bugs are reviewed and their state, owner, and next steps decided.

**Realtime model:** The Azure OpenAI voice model the agent uses to listen and speak (gpt-realtime).

**Loopback audio:** Windows system audio capture - what is playing through the speakers - used to hear remote participants.

**Optimistic concurrency:** Writing only if the item's revision has not changed since it was read.

**Investigation status:** A custom ADO field where dated engineering notes are appended.

**Sub-status:** A triage sub-state such as Investigating or Waiting on partner.

**Milestone:** A release checkpoint - VR (Validation Ready), CC (Code Complete), QCC (Quality Control Check).

**Ephemeral key:** A short-lived credential minted by the backend for the realtime session.





































